

Software Engineering Notes

1. Introduction to Software Engineering

Software Engineering is the systematic approach to designing, developing, testing, deploying, and maintaining software systems.

The main goal of software engineering is to develop reliable, efficient, maintainable, and high-quality software that satisfies user requirements.

Main Activities

Software development normally includes the following activities:

- Requirements analysis
 - System design
 - Implementation
 - Testing
 - Deployment
 - Maintenance
-

2. Software Development Life Cycle

The Software Development Life Cycle (SDLC) is a structured process used to develop software.

Main SDLC Phases

1. Requirements Analysis

In this phase, developers and analysts identify what the customer needs. Functional and non-functional requirements are collected and documented.

2. System Design

The system architecture, database, user interface, and other technical components are designed.

3. Implementation

Developers write the source code based on the system design.

4. Testing

The software is tested to identify defects and verify that it satisfies the requirements.

5. Deployment

The completed software is released and made available to users.

6. Maintenance

After deployment, software is maintained by fixing defects, improving performance, adapting it to new environments, and adding new features.

3. Functional and Non-Functional Requirements

Functional Requirements

Functional requirements describe what a system should do.

Examples:

- The system should allow users to register.
- The system should allow users to log in.
- The system should generate monthly reports.
- The system should allow administrators to delete users.

Non-Functional Requirements

Non-functional requirements describe how the system should perform.

Examples:

- The system should respond within two seconds.
 - The system should be available 99.9% of the time.
 - User passwords should be securely stored.
 - The system should support 1,000 concurrent users.
-

4. Software Testing

Software testing is the process of evaluating software to identify defects and verify that the software meets specified requirements.

Testing helps improve software quality and reliability.

Common Types of Testing

Unit Testing

Unit testing tests individual components or functions of a software system.

Integration Testing

Integration testing checks whether multiple components work correctly together.

System Testing

System testing evaluates the complete software system against its requirements.

Acceptance Testing

Acceptance testing determines whether the software is acceptable to the customer or end user.

Regression Testing

Regression testing ensures that changes to software have not introduced new problems into previously working functionality.

5. Verification and Validation

Verification and Validation are important software quality activities.

Verification

Verification checks whether the software is being developed correctly according to specifications.

Examples include:

- Requirement reviews
- Design reviews
- Code reviews
- Inspections

A common question for verification is:

"Are we building the product right?"

Validation

Validation checks whether the developed software satisfies the actual needs of the user.

Examples include:

- System testing
- Acceptance testing
- User testing

A common question for validation is:

"Are we building the right product?"

6. Software Maintenance

Software maintenance is the process of modifying software after it has been delivered.

Types of Maintenance

Corrective Maintenance

Corrective maintenance fixes defects discovered after software deployment.

Adaptive Maintenance

Adaptive maintenance modifies software so that it works in a changed environment.

For example, software may need modification when the operating system or database platform changes.

Perfective Maintenance

Perfective maintenance improves performance, usability, or functionality.

Preventive Maintenance

Preventive maintenance makes changes that reduce the possibility of future problems.

7. Software Quality

Software quality refers to how well software satisfies specified requirements and user expectations.

Important software quality characteristics include:

- Reliability
- Usability

- Performance
- Security
- Maintainability
- Portability
- Scalability

Reliability

Reliability is the ability of software to perform correctly and consistently for a specified period.

Usability

Usability describes how easily users can learn and use the software.

Maintainability

Maintainability is the ease with which software can be modified, corrected, or improved.

Security

Security protects software and its data from unauthorized access, modification, or destruction.

8. Software Development Methodologies

A software development methodology provides a structured approach for developing software.

Waterfall Model

The Waterfall model follows sequential development phases.

Typical phases include:

Requirements → Design → Implementation → Testing → Deployment → Maintenance

The next phase normally begins after the previous phase has been completed.

Agile Development

Agile is an iterative and incremental approach to software development.

Agile focuses on:

- Customer collaboration
- Continuous feedback

- Frequent delivery
- Responding to change
- Team collaboration

Software is developed in small iterations, allowing requirements to evolve during development.

9. Version Control

Version control systems help developers manage changes to source code and other project files.

Git is a popular distributed version control system.

Common Git Concepts

Repository

A repository stores project files and their version history.

Commit

A commit records a set of changes in the repository.

Branch

A branch allows developers to work on different features or changes independently.

Merge

Merge combines changes from different branches.

Clone

Clone creates a local copy of a remote repository.

10. Database Management

A database is an organized collection of data.

A Database Management System (DBMS) is software used to store, manage, retrieve, and manipulate data.

Examples of relational database systems include:

- MySQL

- PostgreSQL
- Oracle Database
- Microsoft SQL Server

Basic Database Operations

The four basic CRUD operations are:

Create – Add new data.

Read – Retrieve existing data.

Update – Modify existing data.

Delete – Remove data.

11. Application Security

Application security involves protecting software applications from threats and vulnerabilities.

Common security practices include:

- Using strong authentication
- Encrypting sensitive information
- Validating user input
- Using secure communication
- Applying software updates
- Implementing proper access control

Authentication

Authentication verifies the identity of a user.

For example, a username and password can be used to authenticate a user.

Authorization

Authorization determines what an authenticated user is allowed to access.

For example, an administrator may have permission to delete users while a normal user does not.

12. Summary

Software engineering provides systematic methods for developing and maintaining software.

Important concepts include:

- SDLC
- Requirements
- Software design
- Programming
- Testing
- Verification and validation
- Maintenance
- Software quality
- Agile development
- Version control
- Database management
- Application security

A good software engineering process helps organizations produce software that is reliable, secure, maintainable, and useful to its users.